

1.1.1

1. 导入库

```
import pandas as pd
import numpy as np
```

- 导入pandas库并简写为pd，用于数据处理和分析
- 导入numpy库并简写为np，用于数值计算

2. 读取数据

```
data = pd.read_csv('patient_data.csv')
```

- 使用pandas的read_csv函数读取名为'patient_data.csv'的CSV文件
- 将数据存储在名为data的DataFrame中

3. 风险等级分析

```
data['RiskLevel'] = np.where(data['DaysInHospital'] > 7, '高风险患者', '低风险患者')
```

- 使用numpy的where函数创建新列'RiskLevel'
- 如果'DaysInHospital'(住院天数)大于7天，标记为"高风险患者"
- 否则标记为"低风险患者"

```
risk_counts = data['RiskLevel'].value_counts()
```

- 计算不同风险等级的患者数量
- value_counts()返回各分类的计数

```
high_risk_ratio = risk_counts['高风险患者'] / len(data)
low_risk_ratio = risk_counts['低风险患者'] / len(data)
```

- 计算高风险患者占总患者数的比例
- 计算低风险患者占总患者数的比例

```
print("高风险患者数量:", risk_counts['高风险患者'])
print("低风险患者数量:", risk_counts['低风险患者'])
print("高风险患者占比:", high_risk_ratio)
print("低风险患者占比:", low_risk_ratio)
```

- 打印高风险患者数量
- 打印低风险患者数量
- 打印高风险患者占比

- 打印低风险患者占比

4. BMI区间分析

```
bmi_bins = [0, 18.5, 24, 28, np.inf]
bmi_labels = ['偏瘦', '正常', '超重', '肥胖']
```

- 定义BMI区间的边界点：0-18.5(偏瘦), 18.5-24(正常), 24-28(超重), 28及以上(肥胖)
- 定义对应的区间标签

```
data['BMIRange'] = pd.cut(data['BMI'], bins=bmi_bins, labels=bmi_labels, right=False)
```

- 使用pd.cut将BMI值划分到指定区间
- bins参数指定区间边界
- labels参数指定区间标签
- right=False表示使用左闭右开区间

```
bmi_risk_rate = data.groupby('BMIRange')['RiskLevel'].apply(lambda x: (x == '高风险患者').mean())
```

- 按BMI区间分组
- 计算每个区间内高风险患者的比例
- 使用lambda函数计算每个组中'高风险患者'的均值(比例)

```
bmi_patient_count = data['BMIRange'].value_counts()
```

- 统计每个BMI区间的患者总数

```
print("BMI区间中高风险患者的比例和患者数:")
print(bmi_risk_rate)
print(bmi_patient_count)
```

- 打印各BMI区间高风险患者比例
- 打印各BMI区间患者数量

5. 年龄区间分析

```
age_bins = [0, 26, 36, 46, 56, 66, np.inf]
age_labels = ['≤25岁', '26-35岁', '36-45岁', '46-55岁', '56-65岁', '>65岁']
```

- 定义年龄区间的边界点
- 定义对应的年龄区间标签

```
data['AgeRange'] = pd.cut(data['Age'], bins=age_bins, labels=age_labels, right=False)
```

- 使用pd.cut将年龄划分到指定区间
- 参数含义与BMI区间划分相同

```
age_risk_rate = data.groupby('AgeRange')['RiskLevel'].apply(lambda x: (x == '高风险患者').mean())
```

- 按年龄区间分组
- 计算每个区间内高风险患者的比例

```
age_patient_count = data['AgeRange'].value_counts()
```

- 统计每个年龄区间的患者总数

```
print("年龄区间中高风险患者的比例和患者数:")  
print(age_risk_rate)  
print(age_patient_count)
```

- 打印各年龄区间高风险患者比例
- 打印各年龄区间患者数量

1.1.2

1. 导入库

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

- pandas 用于数据处理和分析，简称为 pd
- numpy 用于数值计算，简称为 np
- matplotlib.pyplot 用于数据可视化，简称为 plt（虽然代码中未实际使用绘图功能）

2. 读取数据

```
data = pd.read_csv('sensor_data.csv')
```

- 使用 pandas 的 read_csv 函数读取名为 'sensor_data.csv' 的传感器数据文件
- 将数据存储在名为 data 的 DataFrame 中

3. 传感器数据统计

```
sensor_stats = data.groupby('SensorType')['Value'].agg(['count', 'mean'])
```

- 按 'SensorType'（传感器类型）对数据进行分组

- 对每组中的 'value' (数值) 列进行聚合计算
- `agg(['count', 'mean'])` 计算每个传感器类型的数据数量和平均值
- 结果存储在 `sensor_stats` 变量中

```
print("传感器数据数量和平均值:")
print(sensor_stats)
```

- 打印标题
- 打印统计结果

4. 按位置统计温度和湿度数据

```
location_stats = data[data['SensorType'].isin(['Temperature',
'Humidity'])].groupby(['Location', 'SensorType'])['value'].mean().unstack()
```

- `data['SensorType'].isin(['Temperature', 'Humidity'])` 筛选出传感器类型为温度或湿度的数据
- `.groupby(['Location', 'SensorType'])` 按位置和传感器类型双重分组
- `['value'].mean()` 计算每组数值的平均值
- `.unstack()` 将分组结果转换为更易读的表格形式 (将 `SensorType` 从行索引转为列名)
- 结果存储在 `location_stats` 变量中

```
print("每个位置的温度和湿度数据平均值:")
print(location_stats)
```

- 打印标题
- 打印统计结果

5. 数据清洗和异常值处理

```
data['is_abnormal'] = np.where(
    ((data['SensorType'] == 'Temperature') & ((data['value'] < -10) | (data['value'] > 50)))
    |
    ((data['SensorType'] == 'Humidity') & ((data['value'] < 0) | (data['value'] > 100))),
    True, False
)
```

- 创建新列 `'is_abnormal'` 标记异常值
- 使用 `numpy.where` 进行条件判断:
 - 如果是温度传感器且数值 < -10 或 > 50
 - 或者是湿度传感器且数值 < 0 或 > 100
- 满足条件标记为 `True` (异常), 否则为 `False` (正常)

```
print("异常值数量:", data['is_abnormal'].sum())
```

- 统计并打印异常值数量（`True` 会被视为 1，`False` 为 0，所以 `sum()` 可以计算异常值数量）

6. 填补缺失值

```
data['value'].fillna(method='ffill', inplace=True)
```

- 使用前向填充（forward fill）方法填补缺失值（NaN）
- 用前一个有效值填充后面的缺失值
- `inplace=True` 表示直接修改原 DataFrame

```
data['value'].fillna(method='bfill', inplace=True)
```

- 使用后向填充（backward fill）方法填补剩余的缺失值
- 用后一个有效值填充前面的缺失值
- 这样可以确保所有缺失值都被填补（如果开头有缺失值，`ffill` 无法处理）

7. 保存清洗后的数据

```
cleaned_data = data.drop(columns=['is_abnormal'])
```

- 创建清洗后的数据集 `cleaned_data`
- 删除用于标记异常值的列 `'is_abnormal'`

```
cleaned_data.to_csv('cleaned_sensor_data.csv', index=False)
```

- 将清洗后的数据保存到新的 CSV 文件 `'cleaned_sensor_data.csv'`
- `index=False` 表示不保存行索引

```
print("数据清洗完成，已保存为 'cleaned_sensor_data.csv'")
```

- 打印完成信息

1.1.3

1. 导入库

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

- `pandas` 用于数据处理和分析，简称为 `pd`
- `numpy` 用于数值计算，简称为 `np`

- `matplotlib.pyplot` 用于数据可视化，简写为 `plt`（虽然代码中未实际使用绘图功能）

2. 读取数据

```
data = pd.read_csv('credit_data.csv')
```

- 使用 `pandas` 的 `read_csv` 函数读取名为 `'credit_data.csv'` 的信用数据文件
- 将数据存储在名为 `data` 的 `DataFrame` 中

3. 数据完整性审核

```
missing_values = data.isnull().sum()
```

- `data.isnull()` 检测 `DataFrame` 中的缺失值（NaN），返回布尔值 `DataFrame`
- `.sum()` 计算每列的缺失值数量
- 结果存储在 `missing_values` 变量中

```
duplicate_values = data.duplicated().sum()
```

- `data.duplicated()` 检测重复行，返回布尔值 `Series`
- `.sum()` 计算重复行的总数
- 结果存储在 `duplicate_values` 变量中

```
print("缺失值统计:")  
print(missing_values)  
print("重复值统计:")  
print(duplicate_values)
```

- 打印缺失值统计结果
- 打印重复值统计结果

4. 数据合理性审核

```
data['is_age_valid'] = data['Age'].between(18, 70)
```

- 创建新列 `'is_age_valid'` 标记年龄是否合理
- `between(18, 70)` 检查年龄是否在 18-70 岁之间（合理范围）
- 合理返回 `True`，否则返回 `False`

```
data['is_income_valid'] = data['Income'] > 2000
```

- 创建新列 `'is_income_valid'` 标记收入是否合理
- 检查收入是否大于 2000（最低合理收入）

```
data['is_loan_amount_valid'] = data['LoanAmount'] < (data['Income'] * 5)
```

- 创建新列 'is_loan_amount_valid' 标记贷款金额是否合理
- 检查贷款金额是否小于收入的 5 倍（合理负债比例）

```
data['is_credit_score_valid'] = data['CreditScore'].between(300, 850)
```

- 创建新列 'is_credit_score_valid' 标记信用评分是否合理
- 检查信用评分是否在 300-850 之间（FICO 信用评分标准范围）

5. 合理性检查结果

```
validity_checks = data[['is_age_valid', 'is_income_valid', 'is_loan_amount_valid',  
'is_credit_score_valid']].all(axis=1)
```

- 从 DataFrame 中选取四个合理性检查列
- `.all(axis=1)` 检查每行是否所有条件都为 `True`（即所有字段都合理）
- 结果存储在 `validity_checks` 变量中（布尔值 Series）

```
data['is_valid'] = validity_checks
```

- 创建新列 'is_valid' 标记整行数据是否完全合理

```
print("数据合理性检查:")  
print(data[['is_age_valid', 'is_income_valid', 'is_loan_amount_valid',  
'is_credit_score_valid', 'is_valid']].describe())
```

- 打印合理性检查结果的统计摘要
- `describe()` 提供计数、唯一值、频率等统计信息

6. 数据清洗和异常值处理

```
invalid_rows = data[~data['is_valid']]
```

- 筛选出不合理的数据行（`is_valid` 为 `False` 的行）
- `~` 表示逻辑取反
- 结果存储在 `invalid_rows` 变量中

```
cleaned_data = data[data['is_valid']]
```

- 筛选出合理的数据行（`is_valid` 为 `True` 的行）
- 结果存储在 `cleaned_data` 变量中

```
cleaned_data = cleaned_data.drop(columns=['is_age_valid', 'is_income_valid',
'is_loan_amount_valid', 'is_credit_score_valid', 'is_valid'])
```

- 从清洗后的数据中删除所有用于标记的临时列
- `drop(columns=[...])` 删除指定列

```
cleaned_data.to_csv('cleaned_credit_data.csv', index=False)
```

- 将清洗后的数据保存到新的 CSV 文件 `'cleaned_credit_data.csv'`
- `index=False` 表示不保存行索引

```
print("数据清洗完成，已保存为 'cleaned_credit_data.csv'")
```

- 打印完成信息

1.1.4

1. 导入库

```
import pandas
import numpy as np
import matplotlib.pyplot as plt
```

- `pandas` 用于数据处理和分析（这里没有使用常见的缩写 `pd`）
- `numpy` 用于数值计算，简称为 `np`
- `matplotlib.pyplot` 用于数据可视化，简称为 `plt`（虽然代码中未实际使用绘图功能）

2. 数据采集

```
data = pandas.read_csv('user_behavior_data.csv')
```

- 使用 `pandas` 的 `read_csv` 函数读取名为 `'user_behavior_data.csv'` 的用户行为数据文件
- 将数据存储在名为 `data` 的 `DataFrame` 中

```
print("数据采集完成，已加载到DataFrame中")
```

- 打印数据加载完成的信息

```
print(data.head())
```

- 使用 `head()` 方法打印 `DataFrame` 的前5行数据
- 用于快速查看数据结构和内容

3. 数据清洗与预处理

```
data = data.dropna()
```

- 使用 `dropna()` 方法删除包含缺失值 (NaN) 的行
- 直接修改原 DataFrame

```
data['Age'] = data['Age'].astype(int)
```

- 将 `Age` 列的数据类型转换为整数型 (int)
- 确保年龄数据为整数

```
data['PurchaseAmount'] = data['PurchaseAmount'].astype(float)
```

- 将 `PurchaseAmount` 列的数据类型转换为浮点型 (float)
- 确保购买金额可以有小数

```
data['ReviewScore'] = data['ReviewScore'].astype(int)
```

- 将 `ReviewScore` 列的数据类型转换为整数型 (int)
- 确保评分数据为整数

```
data = data[(data['Age'].between(18, 70)) &  
            (data['PurchaseAmount'] > 0) &  
            (data['ReviewScore'].between(1, 5))]
```

- 筛选保留合理范围内的数据：
 - 年龄在18-70岁之间
 - 购买金额大于0
 - 评分在1-5分之间
- 使用布尔索引过滤不合理数据

```
data['PurchaseAmount'] = (data['PurchaseAmount'] - data['PurchaseAmount'].mean()) /  
data['PurchaseAmount'].std()
```

- 对 `PurchaseAmount` 列进行标准化 (Z-score标准化)
- 公式: (原始值 - 平均值) / 标准差
- 使数据均值为0, 标准差为1

```
data['ReviewScore'] = (data['ReviewScore'] - data['ReviewScore'].mean()) /  
data['ReviewScore'].std()
```

- 对 `ReviewScore` 列进行同样的标准化处理

```
data.to_csv('cleaned_user_behavior_data.csv', index=False)
```

- 将清洗后的数据保存到新的CSV文件 'cleaned_user_behavior_data.csv'
- `index=False` 表示不保存行索引

```
print("数据清洗完成，已保存为 'cleaned_user_behavior_data.csv'")
```

- 打印数据清洗完成的信息

4. 数据统计

```
purchase_category_counts = data['PurchaseCategory'].value_counts()
```

- 统计 `PurchaseCategory` 列中各购买类别的用户数量
- `value_counts()` 返回各分类的计数
- 结果存储在 `purchase_category_counts` 变量中

```
print("每个购买类别的用户数:\n", purchase_category_counts)
```

- 打印购买类别统计结果

```
gender_purchase_amount_mean = data.groupby('Gender')['PurchaseAmount'].mean()
```

- 按 `Gender` 列分组
- 计算每组中 `PurchaseAmount` 的平均值
- 结果存储在 `gender_purchase_amount_mean` 变量中

```
print("不同性别的平均购买金额:\n", gender_purchase_amount_mean)
```

- 打印不同性别的平均购买金额

```
bins = [18, 26, 36, 46, 56, 66, np.inf]
labels = ['18-25', '26-35', '36-45', '46-55', '56-65', '65+']
```

- 定义年龄区间的边界点
- 定义对应的年龄组标签

```
data['AgeGroup'] = pandas.cut(data['Age'], bins=bins, labels=labels, right=False)
```

- 使用 `cut()` 方法将年龄划分到指定区间
- `bins` 参数指定区间边界
- `labels` 参数指定区间标签
- `right=False` 表示使用左闭右开区间

- 结果存储在新列 'AgeGroup' 中

```
age_group_counts = data['AgeGroup'].value_counts().sort_index()
```

- 统计各年龄组的用户数量
- `value_counts()` 计算各组的数量
- `sort_index()` 按索引（年龄组标签）排序
- 结果存储在 `age_group_counts` 变量中

```
print("不同年龄段的用户数:\n", age_group_counts)
```

- 打印各年龄段的用户数量统计结果

1.1.5

1. 导入库

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

- `pandas` 用于数据处理和分析，简称为 `pd`
- `numpy` 用于数值计算，简称为 `np`
- `matplotlib.pyplot` 用于数据可视化，简称为 `plt`（虽然代码中未实际使用绘图功能）

2. 数据采集

```
data = pd.read_csv('vehicle_traffic_data.csv')
```

- 使用 `pandas` 的 `read_csv` 函数读取名为 'vehicle_traffic_data.csv' 的车辆交通数据文件
- 将数据存储在名为 `data` 的 DataFrame 中

```
print("数据采集完成，已加载到DataFrame中")
```

- 打印数据加载完成的信息

```
print(data.head())
```

- 使用 `head()` 方法打印 DataFrame 的前5行数据
- 用于快速查看数据结构和内容

3. 数据清洗与预处理

```
data = data.dropna()
```

- 使用 `dropna()` 方法删除包含缺失值 (NaN) 的行
- 直接修改原 DataFrame

```
data['Age'] = data['Age'].astype(int)
```

- 将 `Age` 列的数据类型转换为整数型 (int)
- 确保年龄数据为整数

```
data['Speed'] = data['Speed'].astype(float)
```

- 将 `Speed` 列的数据类型转换为浮点型 (float)
- 确保车速可以有小数

```
data['TravelDistance'] = data['TravelDistance'].astype(float)
```

- 将 `TravelDistance` 列的数据类型转换为浮点型 (float)
- 确保行驶距离可以有小数

```
data['TravelTime'] = data['TravelTime'].astype(float)
```

- 将 `TravelTime` 列的数据类型转换为浮点型 (float)
- 确保行驶时间可以有小数

```
data = data[(data['Age'].between(18, 70)) &  
            (data['Speed'].between(0, 200)) &  
            (data['TravelDistance'].between(1, 1000)) &  
            (data['TravelTime'].between(1, 1440))]
```

- 筛选保留合理范围内的数据：
 - 年龄在18-70岁之间
 - 车速在0-200 km/h之间
 - 行驶距离在1-1000公里之间
 - 行驶时间在1-1440分钟 (24小时) 之间
- 使用布尔索引过滤不合理数据

```
data.to_csv('cleaned_vehicle_traffic_data.csv', index=False)
```

- 将清洗后的数据保存到新的CSV文件 `'cleaned_vehicle_traffic_data.csv'`
- `index=False` 表示不保存行索引

```
print("数据清洗完成，已保存为 'cleaned_vehicle_traffic_data.csv'")
```

- 打印数据清洗完成的信息

4. 数据合理性审核

```
unreasonable_data = data[~((data['Age'].between(18, 70)) &  
                             (data['Speed'].between(0, 200)) &  
                             (data['TravelDistance'].between(1, 1000)) &  
                             (data['TravelTime'].between(1, 1440)))]
```

- 筛选出不合理的数据行（不符合合理性条件的行）
- `~` 表示逻辑取反
- 结果存储在 `unreasonable_data` 变量中

```
print("不合理的数据:\n", unreasonable_data)
```

- 打印不合理的数据

5. 数据统计

```
traffic_event_counts = data['TrafficEvent'].value_counts()
```

- 统计 `TrafficEvent` 列中各交通事件的发生次数
- `value_counts()` 返回各分类的计数
- 结果存储在 `traffic_event_counts` 变量中

```
print("每种交通事件的发生次数:\n", traffic_event_counts)
```

- 打印交通事件统计结果

```
gender_stats = data.groupby('Gender').agg({'Speed': 'mean', 'TravelDistance': 'mean',  
                                           'TravelTime': 'mean'})
```

- 按 `Gender` 列分组
- 计算每组中 `Speed`、`TravelDistance` 和 `TravelTime` 的平均值
- `agg()` 方法指定要计算的统计量
- 结果存储在 `gender_stats` 变量中

```
print("不同性别的平均车速、行驶距离和行驶时间:\n", gender_stats)
```

- 打印不同性别的统计结果

```
age_bins = [18, 26, 36, 46, 56, 66, np.inf]
age_labels = ['18-25', '26-35', '36-45', '46-55', '56-65', '65+']
```

- 定义年龄区间的边界点
- 定义对应的年龄组标签

```
data['AgeGroup'] = pd.cut(data['Age'], bins=age_bins, labels=age_labels, right=False)
```

- 使用 `cut()` 方法将年龄划分到指定区间
- `bins` 参数指定区间边界
- `labels` 参数指定区间标签
- `right=False` 表示使用左闭右开区间
- 结果存储在新列 `'AgeGroup'` 中

```
age_group_counts = data['AgeGroup'].value_counts()
```

- 统计各年龄组的驾驶员数量
- `value_counts()` 计算各组的数量
- 结果存储在 `age_group_counts` 变量中

```
print("不同年龄段的驾驶员数:\n", age_group_counts)
```

- 打印各年龄段的驾驶员数量统计结果